

Guia de Construção do Zero

Como reconstruir, com as suas próprias mãos, o sistema de Recomendação de Refatorações via Debate Multiagente

Este documento é um **roteiro de construção**, não um "como rodar". Ele descreve, em ordem, a sequência exata de etapas que segui para construir o projeto inteiro — fase por fase, com a decisão de design por trás de cada uma e um *checkpoint* para você confirmar que aquela camada funciona antes de seguir para a próxima.

A ideia é que você **construa você mesmo**, do zero. Em cada fase eu digo *o que criar, por que, quais conceitos estão em jogo e como verificar*. Para os trechos mais delicados, mostro o esqueleto da interface — mas o recheio é seu. Quando travar, o código de referência do repositório está apontado em cada fase.

No final há uma **lista de estudo** organizada por tema, na ordem em que faz sentido aprender.

Índice

1. Filosofia: construa em camadas e verifique sempre
2. Preparando o ambiente
3. O mapa mental da arquitetura
4. A sequência de construção (12 fases) - Fase 0 — Esqueleto do projeto - Fase 1 — Configuração e bootstrap - Fase 2 — Núcleo: AST + métricas - Fase 3 — Camada de ferramentas - Fase 4 — Provedor de LLM com fallback - Fase 5 — Modelos de domínio do debate - Fase 6 — Agentes especialistas - Fase 7 — Agente Juiz e protocolo de debate - Fase 8 — Orquestrador (o pipeline) - Fase 9 — API REST e CLI - Fase 10 — Testes - Fase 11 — Exemplo e documentação - Fase 12 — Harness de avaliação
5. Armadilhas reais que encontrei
6. O que estudar — currículo
7. Roteiro de estudo sugerido

1. Filosofia: construa em camadas e verifique sempre

Três princípios que tornam um projeto deste tamanho gerenciável:

Esqueleto andante (*walking skeleton*) primeiro. Faça o sistema atravessar de ponta a ponta o mais cedo possível — mesmo que cada peça seja simples. Aqui, isso significa: primeiro faça funcionar em **modo heurístico** (sem LLM, sem ferramentas pesadas), com o fluxo inteiro rodando. Só depois adicione o LLM, depois a análise dinâmica, etc. Assim você nunca fica semanas sem nada funcionando.

Construa de baixo para cima, na ordem das dependências. Uma camada só depende das que vieram antes. Config → núcleo → ferramentas → LLM → modelos do debate → agentes → protocolo → orquestrador → API. Se você seguir essa ordem, cada peça nova sempre tem suas dependências prontas.

Checkpoint a cada fase. Nunca passe para a fase seguinte sem rodar algo que prove que a atual funciona (um `python -c`, um teste, um `curl`). Erros pegos cedo custam minutos; erros pegos tarde custam horas.

Dica de ouro: os 13 commits do projeto são exatamente estas fases, na ordem. Rode `git log --oneline --reverse` para ver a trilha. Você pode reconstruir commit a commit.

2. Preparando o ambiente

```
# 1. Instale o uv (gerencia Python + dependências; não precisa instalar Python antes)
curl -LsSf https://astral.sh/uv/install.sh | sh

# 2. Crie a pasta do projeto e inicie o Git
mkdir ai-multi-agent-refactoring-debate
cd ai-multi-agent-refactoring-debate
git init
```

Por que `uv` e não `pip/venv` direto? Porque ele resolve Python + dependências de forma rápida e reproduzível, e instala a versão certa de Python sozinho. Aqui usamos **Python 3.12** (não 3.13 — explico o porquê na seção de armadilhas).

3. O mapa mental da arquitetura

Antes de escrever uma linha, tenha clara a figura. O sistema tem **duas camadas** e um **fluxo linear**:

```
código.py
▼
[1] AST parser ████████████████████ representação sintática
    ▼
[2] CAMADA DE FERRAMENTAS (determinística)
    ■■ estáticas: Radon, Pylint, import-linter, SonarQube
    ■■ dinâmicas: Scalene, py-spy, cProfile
    ■■ energia:   CodeCarbon
    ■ (tudo normalizado num "JSON unificado" de métricas)
    ▼
[3] CAMADA DE AGENTES (raciocínio)
    ■■ Agente de Sustentabilidade ← Radon, Scalene, CodeCarbon
    ■■ Agente de Arquitetura      ← SonarQube, Pylint, import-linter
    ■■ Agente de Desempenho       ← Scalene, py-spy, cProfile
    ■ (cada um vê SÓ a fatia de métricas da sua área)
    ▼
[4] DEBATE: cada especialista critica os outros (revisão por pares)
    ▼
[5] AGENTE JUIZ: detecta conflitos → negocia trade-offs → consolida
    ▼
[6] saída consolidada (via CLI ou API REST)
```

Guarde duas ideias-chave:

- **Separação ferramentas × agentes.** Ferramentas produzem *números* determinísticos. Agentes produzem *recomendações* a partir desses números. Isso mantém o raciocínio do LLM "ancorado" em evidências.
- **Roteamento por escopo.** Cada agente recebe apenas as ferramentas da sua dimensão. É isso que cria a especialização — e, conseqüentemente, os conflitos do debate.

4. A sequência de construção (12 fases)

Cada fase abaixo segue o mesmo formato: ■ **Objetivo** · ■ **O que criar** · ■ **Conceitos** · ■ **Decisões** · ■ **Checkpoint**.

Fase 0 — Esqueleto do projeto · chore: scaffold

■ **Objetivo:** ter um projeto Python instalável, com a árvore de pastas e a configuração de build, antes de qualquer lógica.

■ **O que criar:** - `pyproject.toml` — metadados, dependências, `requires-python = ">=3.12,<3.13"`, build com hatchling, layout `src/`. Defina já o `script` de console (`refactoring-debate = "..."`). - `README.md` — **precisa existir** (o build do hatchling lê o `readme`), nem que seja um rascunho. - `.gitignore` — `.env/`, `__pycache__`/, `runs/`, `emissions.csv`, `caches`. - `.env.example` — modelo de configuração (todas as variáveis com prefixo `RD_`). - `src/refactoring_debate/__init__.py` — com `__version__`.

■ **Conceitos:** empacotamento Python moderno (PEP 621), layout `src/`, *console scripts*, ambientes virtuais.

■ **Decisões:** - Layout `src/` (e não pacote na raiz) evita importar acidentalmente o código não instalado e força você a testar o pacote "de verdade". - Toda configuração via variáveis de ambiente com prefixo (`RD_`) — facilita rodar em servidores e em CI.

■ **Checkpoint:**

```
uv sync                                # cria a venv 3.12 e instala
uv run python -c "import refactoring_debate; print(refactoring_debate.__version__)"
```

Se imprimiu a versão, o pacote está instalável. (Referência: `pyproject.toml`.)

Fase 1 — Configuração e bootstrap · feat: settings and bootstrap

■ **Objetivo:** um objeto central de configuração e a preparação do processo (logging, desligar telemetria de bibliotecas).

■ **O que criar:** - `config.py` — uma classe `Settings(BaseSettings)` do `pydantic-settings`, lendo `.env` com `env_prefix="RD_"`. Campos: provedor de LLM, modelo, pesos do juiz, timeouts, flags de análise dinâmica, etc. Adicione uma função `get_settings()` com cache. - `bootstrap.py` — funções para configurar logging (loguru) e variáveis de ambiente que silenciam telemetria de terceiros (CrewAI, etc.). **Isso precisa rodar antes de importar o CrewAI.**

■ **Conceitos:** Pydantic v2, `pydantic-settings`, `Enum`, `@lru_cache`, doze-fatores (config no ambiente).

■ **Decisões:** um `Enum LLMProvider` com um valor `heuristic` — esse é o segredo que permite o sistema rodar sem LLM nenhum.

■ **Checkpoint:**

```
uv run python -c "from refactoring_debate.config import get_settings; print(get_settings().decision_weights)"
```

(Referência: `config.py`, `bootstrap.py`.)

Fase 2 — Núcleo: AST + métricas · feat: AST parser and metrics model

■ **Objetivo:** transformar código-fonte em (a) uma representação estrutural e (b) um formato unificado para guardar as métricas de qualquer ferramenta.

■ **O que criar:** - `core/ast_parser.py` — usa o módulo `ast` da biblioteca padrão para extrair de cada função: nº de argumentos, profundidade de laços aninhados, nº de ramos, complexidade estimada, docstrings, etc.; e de cada classe: métodos, bases. Capture erros de sintaxe (não deixe explodir). - `core/metrics.py` — os modelos Pydantic do "JSON unificado": `Severity`, `ToolStatus`, `Finding` (um achado normalizado) e `ToolResult` (saída de uma ferramenta). E um `MetricsReport` com um método `slice(tool_names)` que devolve só as ferramentas de um escopo (é assim que cada agente recebe só a sua fatia).

Esqueleto do modelo central:

```
class ToolResult(BaseModel):
    tool: str
    category: ToolCategory          # static | dynamic | energy
    status: ToolStatus = ToolStatus.OK
    summary: str = ""
    metrics: dict[str, Any] = {}    # números livres da ferramenta
    findings: list[Finding] = []    # achados normalizados
```

■ **Conceitos:** o módulo `ast` (`NodeVisitor`, percorrer árvores), métricas de software (complexidade ciclomática, índice de manutenibilidade), modelagem com Pydantic.

■ **Decisões:** **normalizar tudo num formato comum** (`ToolResult`/`Finding`). Sem isso, cada agente teria que conhecer o formato cru de cada ferramenta — um pesadelo. Com isso, agentes e ferramentas ficam desacoplados.

■ **Checkpoint:** escreva um python `-c` que dá `parse_code("def f(x):\n for ...")` e imprime `max_loop_depth`. (Referência: `core/ast_parser.py`, `core/metrics.py`.)

Fase 3 — Camada de ferramentas · feat: deterministic tools layer

■ **Objetivo:** envelopar cada analisador externo de modo que todos tenham a mesma interface e **degradem com elegância** quando ausentes.

■ **O que criar:** - `tools/base.py` — uma classe abstrata `Analyzer` com um *template method* `run(ctx)` que: checa disponibilidade → cronometra → captura exceções → devolve um `ToolResult`. As subclasses só implementam `analyze(ctx)`. Crie também um `AnalysisContext` (código, caminho do arquivo num diretório temporário, AST, settings) e um `DynamicAnalyzer` base que só roda se a análise dinâmica estiver habilitada. - Comece por **uma** ferramenta fácil: `tools/static/radon_tool.py` (Radon é Python puro, sempre disponível). Depois `pylint_tool.py` (rode via subprocess com saída JSON). Depois as demais, uma por vez. - `tools/registry.py` — a lista de analisadores e o mapa `AGENT_TOOL_SCOPES` (qual ferramenta alimenta qual agente — a Figura 1 do artigo).

Padrão do *template method*:

```
class Analyzer(ABC):
    name: str
    category: ToolCategory
    def run(self, ctx) -> ToolResult:
```

```

status, reason = self.availability(ctx)
if status is not ToolStatus.OK:
    return ToolResult(..., status=status, summary=reason) # degrada
try:
    return self.analyze(ctx) # faz o trabalho
except Exception as e:
    return ToolResult(..., status=ToolStatus.ERROR, error=str(e))
@abstractmethod
def analyze(self, ctx) -> ToolResult: ...

```

■ **Conceitos:** padrão *Template Method*, subprocess com timeout, *graceful degradation*, profiling (cProfile/Scalene), a API de cada ferramenta.

■ **Decisões:** - **Uma ferramenta nunca derruba o pipeline.** Se faltar o binário, ou der erro, ou estiver desligada, ela retorna um `ToolResult` com status `unavailable/skipped/error` e o resto segue. - **Análise dinâmica desligada por padrão** — ela *executa* o código analisado (risco de segurança). Fica atrás de uma flag.

■ **Checkpoint:** monte um `AnalysisContext` e rode `RadonAnalyzer().run(ctx)`; confirme `status == ok` e que há métricas. Vá adicionando ferramentas e re-testando. (Referência: `tools/`.)

Fase 4 — Provedor de LLM com fallback · feat: `pluggable LLM provider`

■ **Objetivo:** uma camada que entrega um "cérebro" para os agentes — Ollama, OpenAI, Anthropic — e que **cai no modo heurístico** se nada estiver disponível.

■ **O que criar:** - `llm/provider.py` — uma função `build_llm(settings)` que devolve um `LLMHandle`. Se o provedor for `ollama`, faça um *probe* (uma requisição ao servidor) para ver se está no ar; se não estiver, retorne um handle heurístico. O `LLMHandle` carrega o objeto LLM do CrewAI (ou `None`) e uma propriedade `uses_llm`.

■ **Conceitos:** abstração de provedor, `LiteLLM/CrewAI LLM`, *health check/probe*, degradação graciosa de novo.

■ **Decisões:** este é o coração da robustez. Os agentes nunca falam com o LLM diretamente — só com o `LLMHandle`. Assim, trocar Llama3 por Claude é configuração, não código; e ficar sem LLM não quebra nada.

■ **Checkpoint:**

```

uv run python -c "from refactoring_debate.config import Settings; from refactoring_debate.llm.provider import build_llm
# -> False (modo heurístico)

```

(Referência: `llm/provider.py`.)

Fase 5 — Modelos de domínio do debate · feat: `debate domain models`

■ **Objetivo:** os tipos de dados que descrevem o debate: recomendações, críticas, conflitos, trade-offs e o resultado final.

■ **O que criar:** - `debate/models.py` — `Dimension` (sustentabilidade/arquitetura/desempenho/juiz), `Recommendation`, `AgentReport`, `Critique`, `Conflict`, `Tradeoff`, `ConsolidatedRecommendation`, `DebateRecord` e o `AnalysisResult` que junta tudo. Inclua um `ResearchMetrics` para os indicadores Q1/Q2/Q3 do artigo.

■ **Conceitos:** *Domain modeling*, herança de modelos Pydantic (`ConsolidatedRecommendation` herda de `Recommendation`), enums de domínio.

■ **Decisões:** modelar o **registro do debate** como dado de primeira classe. É isso que permite, lá no fim, medir Q3 (conflitos) e mostrar a justificativa do juiz.

■ **Checkpoint:** instanciar uma `Recommendation` e serializar com `.model_dump_json()`. (Referência: `debate/models.py`.)

Fase 6 — Agentes especialistas • feat: `specialist and judge agents`

■ **Objetivo:** os três especialistas. Cada um produz recomendações **com LLM** (se houver) ou **por regras determinísticas** (se não houver).

■ **O que criar:** - `agents/base.py` — uma classe `SpecialistAgent` com: - `analyze(ast, metrics)` que fatia as métricas do seu escopo e despacha: se `llm.uses_llm`, monta um agente CrewAI + tarefa e parseia o JSON da resposta; senão, chama `_heuristic_recommendations(...)`. - `critique(others)` que faz a revisão por pares. - `_heuristic_recommendations(...)` **abstrato** — cada especialista implementa as suas regras. - `agents/prompts.py` — instruções para o LLM (pedir JSON) e um extrator de JSON tolerante. - `agents/sustainability_agent.py`, `architecture_agent.py`, `performance_agent.py` — cada um define seu `role/goal/backstory` (persona do CrewAI), seu escopo de ferramentas e suas regras heurísticas.

■ **Conceitos:** CrewAI (Agent/Task/Crew), *prompt engineering*, *structured output* (pedir JSON e parsear com tolerância), padrão *Strategy* (LLM vs heurística), padrão *Template Method* de novo.

■ **Decisões:** - **Todo agente tem dois cérebros.** O caminho heurístico não é só *fallback* — é o que faz o sistema demonstrável sem GPU/LLM, e serve de baseline determinístico. - As regras de desempenho propõem *cache/parallelismo* de propósito — é isso que **cria** os conflitos com sustentabilidade e arquitetura mais adiante.

■ **Checkpoint:** rode um especialista em modo heurístico sobre um AST com laço aninhado e confira que ele recomenda algo. (Referência: `agents/`.)

Fase 7 — Agente Juiz e protocolo de debate • feat: `judge and protocol`

■ **Objetivo:** o mediador que detecta conflitos, arbitra trade-offs com pesos e consolida; e o protocolo que orquestra as rodadas de crítica.

■ **O que criar:** - `agents/judge_agent.py` — `detect_conflicts(...)` (mesmo alvo + dimensões diferentes → conflito; críticas de oposição → conflito), `arbitrate(...)` (favorece a dimensão de maior peso × severidade; rebaixa as outras para "deferred"), `consolidate(...)` (pontua, ordena por prioridade, atribui status) e um `summarize(...)`. - `debate/protocol.py` — uma classe `DebateProtocol` que: coleta as recomendações, roda N rodadas em que cada especialista critica os outros, e então chama o juiz (`detect` → `arbitrate` → `consolidate` → `summarize`).

■ **Conceitos:** Multi-Agent Debate (MAD), teoria de decisão simples (pesos), resolução de conflitos, *trade-offs* como fronteira de Pareto.

■ **Decisões:** a detecção de conflitos é **determinística** (não depende do LLM) — assim Q3 é mensurável e reproduzível mesmo no modo heurístico. O LLM, quando presente, só enriquece o texto.

■ **Checkpoint:** dê ao juiz duas recomendações no mesmo alvo (uma de desempenho com tag `cache`, outra de sustentabilidade) e confirme que ele gera um conflito `performance_vs_sustainability` e rebaixa uma delas. (Referência: `judge_agent.py`, `debate/protocol.py`.)

Fase 8 — Orquestrador (o pipeline) · feat: pipeline orchestrator

■ **Objetivo:** a peça que costura tudo: código → AST → ferramentas → agentes → debate → resultado.

■ **O que criar:** - `core/orchestrator.py` — uma classe `Orchestrator` com `analyze(code, filename)` que executa as fases na ordem, cronometra cada uma, monta o `AnalysisResult`, calcula os indicadores de pesquisa e (opcional) persiste o resultado em JSON.

■ **Conceitos:** padrão *Facade/Pipeline*, gestão de diretório temporário (escrever o código num arquivo para as ferramentas que precisam de caminho), *dependency injection* (receber settings/LLM prontos).

■ **Decisões:** o orquestrador é o **único** lugar que conhece todas as camadas. Tudo abaixo dele é desacoplado. É aqui que o "esqueleto andante" finalmente anda inteiro.

■ **Checkpoint:** este é o grande momento.

```
uv run python -c "from refactoring_debate.config import Settings; from refactoring_debate.core.orchestrator"
```

Se saiu Q1/Q2/Q3, o sistema inteiro funciona de ponta a ponta. (Referência: `core/orchestrator.py`.)

Fase 9 — API REST e CLI · feat: REST API and CLI

■ **Objetivo:** as duas portas de entrada para o usuário.

■ **O que criar:** - `api/schemas.py` — modelos de request/response (`AnalyzeRequest`, etc.). - `api/routes.py` — `POST /api/v1/analyze`, `GET /health`, `GET /api/v1/config`. - `main.py` — a app FastAPI, com um `lifespan` que constrói o `Orchestrator` uma vez no startup e o guarda em `app.state`. - `cli.py` — lê um arquivo, chama o orquestrador e imprime um relatório bonito com `rich`.

■ **Conceitos:** FastAPI (rotas, *dependency injection*, `lifespan`), `argparse`, `rich` (tabelas/painéis no terminal), por que endpoints síncronos pesados rodam em *threadpool*.

■ **Decisões:** construir o orquestrador **uma vez** no startup (o probe do Ollama e a criação dos agentes não devem repetir por requisição).

■ **Checkpoint:**

```
uv run uvicorn refactoring_debate.main:app --reload # abra /docs
uv run refactoring-debate examples/sample_code.py # (crie o exemplo já, ou use qualquer .py)
```

(Referência: `api/`, `main.py`, `cli.py`.)

Fase 10 — Testes · test: test suite

■ **Objetivo:** provar que cada camada funciona e travar regressões.

■ **O que criar:** - `tests/conftest.py` — *fixtures* que forçam o modo heurístico (testes offline, determinísticos, rápidos). - `tests/test_ast_parser.py`, `test_tools.py`, `test_debate.py`, `test_pipeline.py`, `test_api.py` — um arquivo por camada. Para a API, use o `TestClient` do FastAPI.

■ **Conceitos:** `pytest`, *fixtures*, testes determinísticos, `TestClient`, *monkeypatch*.

■ **Decisões:** testar em modo heurístico — sem LLM, sem rede, sem execução de código dinâmico. Testes não podem depender de Ollama no ar.

■ **Checkpoint:** `uv run pytest` — tudo verde.

Fase 11 — Exemplo e documentação · docs: `example + README`

■ **Objetivo:** material de demonstração e o front-door do projeto.

■ **O que criar:** - `examples/sample_code.py` — um módulo com *defeitos propositalis* nas três dimensões (laços aninhados = desempenho/energia; classe enorme = arquitetura; imports sem uso), com um bloco `if __name__ == "__main__":` para a análise dinâmica ter o que medir. - `README.md` completo, `Makefile`, `docker-compose.yml` (SonarQube), `.importlinter` (contratos de arquitetura do próprio projeto).

■ **Conceitos:** *code smells* como fixtures, documentação técnica, automação com Make, import-linter (validar a própria arquitetura de dependências).

■ **Checkpoint:** `uv run lint-imports` — contratos mantidos.

Fase 12 — Harness de avaliação · feat: `evaluation harness`

■ **Objetivo:** as ferramentas para responder cientificamente Q1/Q2/Q3 sobre repositórios reais (o Plano de Validação, Seção 4.3 do artigo).

■ **O que criar:** - `scripts/evaluate.py` — roda o debate sobre cada `.py` de um repositório, calcula um *baseline* de agente único (para Q1) e agrega tudo em **frequências e medianas**. - `scripts/build_tutorial_pdf.py` — gera PDFs a partir de Markdown.

■ **Conceitos:** desenho experimental, estatística por frequências/medianas (e por que **não** médias), baseline de comparação.

■ **Checkpoint:** `uv run python scripts/evaluate.py src/refactoring_debate --baseline`. (Referência: `scripts/evaluate.py`.)

5. Armadilhas reais que encontrei (e como evitá-las)

Coisas que **vão** te travar se você não souber de antemão:

Python 3.13 quebra o CrewAI. A árvore de dependências do CrewAI (e suas dependências de ML) ainda não resolve bem no 3.13. Use **3.12** (`requires-python = ">=3.12,<3.13"`); o `uv` baixa sozinho.

O build falha sem `README.md`. O hatchling lê o campo `readme` do `pyproject.toml`; se o arquivo não existir, `uv sync` falha. Crie o `README` cedo, nem que seja um rascunho.

A CLI do Scalene mudou. Não é `scalene arquivo.py`; é `scalene run --cli --json --outfile X arquivo.py` (subcomando `run`). Quando uma ferramenta externa não responder como você espera, rode o `--help` dela.

py-spy precisa de privilégios no macOS. Ele lê a memória de outro processo, o que o SIP do macOS bloqueia sem `sudo`. Trate o caso "sem permissão" como `unavailable`, não como erro fatal.

CrewAI esconde a Anthropic atrás de um *extra*. Para usar Claude, é preciso `crewai[anthropic]` instalado, senão a construção do LLM falha (e, no nosso design, cai no heurístico). A OpenAI já vem via LiteLLM.

Análise dinâmica executa o código. Scalene/cProfile/CodeCarbon *rodam* o arquivo analisado. Isso é um risco de segurança — deixe desligado por padrão e avise no docs.

Telemetria de terceiros polui o output. Desligue a telemetria do CrewAI por variável de ambiente *antes* de importá-lo (por isso o `bootstrap.py`).

Modelos novos precisam do prefixo de provedor. No LiteLLM, use `anthropic/claude-...` (e `ollama/llama3`), senão ele não sabe rotear modelos recentes.

`ruff --fix` apaga seus *smells* de propósito. Ele removeu os imports sem uso do meu arquivo de exemplo (que eram defeitos intencionais). Exclua a pasta `examples/` do `ruff`.

Estas nove são o tipo de coisa que não está em tutorial nenhum — anote-as.

6. O que estudar — currículo

Organizei por área, com o "porquê" de cada uma e em qual fase ela aparece. Não precisa dominar tudo antes de começar — estude *sob demanda*, na ordem das fases.

A. Python intermediário/avançado (*fases 0–8, a base de tudo*)

- **Type hints e typing** — o projeto inteiro é tipado.
- **O módulo `ast`** — `ast.parse`, `NodeVisitor`, percorrer a árvore. **Essencial** (fase 2).
- **`subprocess`** — rodar ferramentas externas com timeout e capturar saída (fase 3).
- **`dataclasses`, `Enum`, `abc` (classes abstratas)** — padrões de design em Python.
- **Decoradores e `functools` (`lru_cache`)**, gerenciadores de contexto (`with`).
- **Empacotamento moderno** — `pyproject.toml`, PEP 621, layout `src/`, console scripts.

B. Pydantic v2 (*fases 1, 2, 5 — a espinha dorsal de dados*)

- `BaseModel`, validação, `model_dump/model_dump_json`, herança de modelos.
- `pydantic-settings` — configuração por variáveis de ambiente.

C. Análise de software (estática e dinâmica) (*fases 2–3 — o coração técnico*)

- **Complexidade ciclomática, Índice de Manutenibilidade, métricas de Halstead.**
- **Code smells** e dívida técnica (o que são, como se detectam).
- **Profiling**: a diferença entre *profiling* determinístico (cProfile) e por amostragem (py-spy), e profiling de CPU/memória (Scalene).
- **As ferramentas, uma a uma**: Radon, Pylint, import-linter, SonarQube, Scalene, py-spy, cProfile, CodeCarbon — leia o README de cada uma e rode no terminal antes de envelopar.
- **Software verde / green smells** e medição de energia (CodeCarbon).

D. LLMs e sistemas de agentes (*fases 4, 6, 7 — o diferencial do projeto*)

- **Como um LLM é chamado por API** — mensagens, temperatura, *structured output* (pedir JSON e parsear).
- **CrewAI** — Agent, Task, Crew, papéis/personas. Leia a documentação oficial.
- **LiteLLM** — a camada que roteia para Ollama/OpenAI/Anthropic.
- **Ollama** — rodar modelos locais (Llama3).
- **Multi-Agent Systems (MAS)** e **Multi-Agent Debate (MAD)** — o conceito central do artigo: agentes com papéis, argumentação, revisão por pares.

E. Engenharia de software e arquitetura (*transversal*)

- **Refatoração** (Martin Fowler, *Refactoring*) — o domínio do problema.
- **Arquitetura limpa / direção de dependências** — por que `tools` não importa `agents`; o uso do `import-linter` para *garantir* isso.
- **Padrões de projeto** usados aqui: Template Method (ferramentas/agentes), Strategy (LLM vs heurística), Facade (orquestrador), graceful degradation.

F. Web e API (*fase 9*)

- **FastAPI** — rotas, modelos de request/response, *dependency injection*, `lifespan`, documentação automática (Swagger).
- **REST** — verbos, status codes, JSON.
- `rich` e `argparse` — para a CLI.

G. Ferramental e qualidade (*fases 0, 10, 11*)

- `uv` — ambientes e dependências.
- `pytest` — testes e fixtures.
- `ruff` (lint/format) e `mypy` (tipos).
- **Git** — commits atômicos, Conventional Commits, reescrita de histórico.

H. Os artigos de referência (*o "porquê" científico*)

Leia os trabalhos citados no seu próprio artigo: RefAgent (Oueslati et al.), SWE-Debate (Li et al.), MAD para requisitos (Oriol et al.), o framework conceitual (Rajendran et al.), GreenLight (Rajput et al.). Eles fundamentam cada decisão de arquitetura.

7. Roteiro de estudo sugerido (ordem temporal)

Se eu fosse começar do zero hoje, estudaria e construiria **em paralelo**, nesta ordem:

Etapa	Estude	Construa	Resultado
1	Python moderno + <code>uv</code> + <code>pyproject</code>	Fase 0 e 1	projeto instalável + config
2	módulo <code>ast</code> + Pydantic	Fase 2	AST e métricas
3	<code>subprocess</code> + 1 ferramenta (Radon)	Fase 3 (parcial)	primeira métrica real

4	as outras ferramentas, uma a uma	Fase 3 (resto)	camada de ferramentas completa
5	Strategy/Template + heurísticas	Fases 5 e 6 (modo heurístico)	agentes determinísticos
6	MAD + resolução de conflitos	Fases 7 e 8	pipeline andando de ponta a ponta
7	FastAPI + rich	Fase 9	API e CLI
8	pytest	Fase 10	suíte de testes verde
9	CrewAI + Ollama/LiteLLM	Fase 4 + religar agentes ao LLM	agentes com LLM real
10	desenho experimental	Fases 11 e 12	exemplo, docs e avaliação

Repare numa inversão proposital: deixe o **LLM por último** (etapa 9). Construa tudo no modo heurístico primeiro — assim você tem o sistema inteiro funcionando e testável antes de lidar com a parte mais imprevisível (modelos de linguagem). É o princípio do esqueleto andante levado a sério.

Palavra final

O projeto parece grande porque é — mas ele é só **camadas finas, empilhadas na ordem certa**, cada uma verificável sozinha. Se você construir uma fase por vez, testando o checkpoint antes de seguir, nunca vai estar a mais de um passo de algo que funciona. Esse é o segredo: não é talento, é **sequência e verificação**.

Boa construção. Quando travar numa fase, abra o arquivo de referência correspondente no repositório e compare — mas tente sempre escrever a sua versão primeiro.